LiDAR Based SLAM & Textured Occupancy Mapping

1st Lokesh Thangamani
Department of Electrical & Computer Engineering
University of California, San Diego
San Diego, USA
lthangamani@ucsd.edu

***Abstract*—This project implements a Simultaneous Localization and Mapping (SLAM) system for a differential-drive robot. By fusing data from wheel encoders, an Inertial Measurement Unit (IMU), a 2-D LiDAR scanner, and an RGBD camera, we accurately estimate the robot's trajectory. We also concurrently build a representation of its environment shown through an occupancy map. The setup utilizes the Iterative Closest Point (ICP) algorithm for scan matching and constructs a 2-D probabilistic occupancy grid. Using the data from the RGBD camera, a colored floor texture map is also built. We ultimately refine the global trajectory and maps using pose graph optimization with loop closure constraints via the GTSAM library.**

***Index Terms*—Differential Drive Robot, Inertial Measurement Unit, Wheel Encoders, 2D-LiDAR, RGBD, Iterative Closest Point, GTSAM, Loop Closure, Occupancy Map.**

I. Introduction

In the field of autonomous robotics, navigating an unknown environment without external positioning systems is a challenging task. To operate safely, a robot must concurrently build a map of its surroundings and determine its location within that map. This is known as the Simultaneous Localization and Mapping (SLAM) problem. Solving SLAM is important because it is the foundation for most autonomy tasks, which include, but are not limited to, path planning, obstacle avoidance, and reliable interaction with the physical world.

To perceive its own motion and its surroundings, our differential-drive robot relies on suite of sensors. Wheel encoders and an Inertial Measurement Unit (IMU) act as the robot's internal sense of motion, providing reliable data to track immediate changes in velocity and orientation. While essential for dead-reckoning, this pair alone is unreliable as they accumulate drift over time.

To correct this drift and to perceive the outside world, the robot utilizes a 2D-LiDAR and an RGBD camera. LiDAR (Light Detection and Ranging) is data is highly valued in robotics tasks because it provides extremely accurate, low-noise geometric measurements of the environment. This makes it an ideal sensor for recognizing spatial structures surrounding the robot and aligning consecutive observations to correct any trajectory errors/drift. An RGBD camera provides visual and depth information, allowing the robot to add visual texture to the environment.

At a high level, this project follows a 4 phase approach. Odometry Estimation to get baseline estimate of the robot's motion using IMU and Wheel Encoders. Scan Matching to refine this initial trajectory, using LiDAR scans and ICP. Generating an Occupancy map using the LiDAR scans and trajectory information. We also add texture to it using the data from the RGBD sensor. Finally, Trajectory optimization, using loop closures, where we further refine the trajectory.

II. Problem Formulation

A. Encoder & IMU Odometry

Consider a differential-drive robot navigating on a 2-D plane. Let the state of the robot at time t be defined by its pose $x_t = [x_t, y_t, \theta_t]^T$, where (x_t, y_t) represents the position of the robot in the world frame and θ_t represents its orientation (yaw) along the z-axis.

At time t, the robot receives external control inputs $u_t = [v_t, \omega_t]^T$, where $v_t \in \mathbb{R}$ is the linear velocity of the robot, derived from the wheel encoder data and $\omega_t \in \mathbb{R}$ is the angular velocity, acquired from the IMU. Given an initial pose x_0 (origin, $[0, 0, 0]^T$), we iteratively estimate the subsequent states x_{t+1} using the discrete-time differential-drive motion model.

$$x_{t+1} = x_t + \tau_t v_t \cos(\theta_t)$$

$$y_{t+1} = y_t + \tau_t v_t \sin(\theta_t)$$

$$\theta_{t+1} = \theta_t + \tau_t \omega_t$$

here, τ_t represents the time duration between two consecutive sensor measurements t and $t+1$. In this phase, we sequentially apply this motion model $f(x_t, u_t)$ to estimate the full dead-reckoning trajectory before any external sensor corrections are applied.

B. Point Cloud Registration via ICP

In the 3-D ICP warm-up, with the given point cloud and a canonical object model, the objective is to find the optimal 3-D rotation R and translation p. The optimal pose is what minimizes the Euclidean distance between associated points.

For LiDAR scan matching, we perform 2-D point-cloud registration. Let Z_t and Z_{t+1} be two consecutive LiDAR scans with points represesnted in $\mathbb{R}^2$. We estimate the relative pose change ${}_tT_{t+1} \in SE(2)$ between these scans. This is done by iteratively finding data associations between points and solving the optimization objective to for the optimal 2-D rotation R and translation p.

$$\min_{R \in SO(2), p \in \mathbb{R}^2} \sum_i ||m_{c_i} - (Rz_i + p)||_2^2$$

where $z_i \in Z_{t+1}$ is a source point, and $m_{c_i} \in Z_t$ is its corresponding target point. To avoid local optima, the relative transformation is initialized using the trajectory obtained in the previous step. Once the optimal relative transformation ${}_tT_{t+1}$ is found, the robot pose T_{t+1}, is updated using the previous pose T_t.

$$T_{t+1} = T_t \cdot {}_tT_{t+1}$$

C. Occupancy & Texture Mapping

The 2-D environment can be represented by an occupancy map $M \in \mathbb{R}^{N \times N}$, where each cell $m_i \in M$ is modeled as a Bernoulli random variable indicating the presence of an obstacle or free space. Given the scan-matched robot trajectory $x_{0:T}$ and the entirety of LiDAR scans $z_{1:T}$, the objective is to estimate the posterior probability distribution $p(m_i | z_{1:T}, x_{0:T})$ for each cell within the map. We add a +ve Log probability for obstacle and -ve for free space.

We also generate a similar 2-D map but with texture of the floor, defined as $C \in \mathbb{R}^{N \times N \times 3}$ where the 3 stands for RGB channels. Given the RGBD camera's intrinsic matrix K, and disparity values d at pixel (i, j), we map an image pixel to a 3-D point in the camera's optical frame.

$$\begin{bmatrix} X_o \\ Y_o \\ Z_o \end{bmatrix} = K^{-1} \begin{bmatrix} rgbi \\ rgbj \\ 1 \end{bmatrix} \cdot \text{depth}$$

We transform these coordinates to the world frame and plot the corresponding RGB values to generate the texture map.

D. Optimizing Pose Estimates

The trajectory refinement is setup as a factor graph optimization problem, where nodes $X = \{T_1, \ldots, T_K\}$ represent the robot's poses and edges represent relative pose constraints. Loop closures are setup, based on fixed intervals or pose-proximity.

III. Technical Approach

A. Encoder - IMU Motion Model

The wheel encoders and IMU provide measurements at different rates. To synchronize the two, we use the encoder timestamps as our base time reference. For every interval between two consecutive encoder timestamps, we identify all corresponding IMU yaw rate measurements (ω). We then integrate these over their respective IMU time deltas within that window to compute the total heading change, $\Delta\theta_t$, for that specific encoder time step.

The front-right (FR), front-left (FL), rear-right (RR), and rear-left (RL) encoder counts are used to calculate the distances traveled by the right and left wheels.

$$d_{right} = \frac{FR + RR}{2} \cdot 0.0022$$

$$d_{left} = \frac{FL + RL}{2} \cdot 0.0022$$

The average of these two distances gives the robot's displacement.

$$\Delta s_t = \frac{d_{right} + d_{left}}{2}$$

While the base formula relies on linear velocity and angular velocity, if we factor out τ, displacements suffice. With the displacement Δs_t and the heading change $\Delta\theta_t$, the robot's pose is determined.

$$x_{t+1} = x_t + \Delta s_t \frac{\sin(\Delta\theta_t/2)}{\Delta\theta_t/2} \cos\left(\theta_t + \frac{\Delta\theta_t}{2}\right)$$

$$y_{t+1} = y_t + \Delta s_t \frac{\sin(\Delta\theta_t/2)}{\Delta\theta_t/2} \sin\left(\theta_t + \frac{\Delta\theta_t}{2}\right)$$

$$\theta_{t+1} = \theta_t + \Delta\theta_t$$

For purely straight motion ($|\Delta\theta_t| \leq 10^{-10}$), to avoid division by zero, $\frac{\Delta\theta_t}{2}$ are ignored. Doing the same across all measurements gives us the initial trajectory estimate $[x, y, \theta]_{0:T}^T$.

B. Point-Cloud Registration via ICP

1) Warm Up - Object Pose: We implement a 3-D Iterative Closest Point (ICP) algorithm by estimating the poses of canonical objects. The algorithm iteratively alternates between data association and pose transform optimization. To establish the association between the canonical source model and the observed target point clouds, a k-d tree structure.

$$\min_{\boldsymbol{p} \in \mathbb{R}^d, R \in SO(d), \Delta} \sum_{(i,j) \in \Delta} w_{ij} \left\|(R\boldsymbol{z}_j + \boldsymbol{p}) - \boldsymbol{m}_i\right\|_2^2$$

$$i \leftrightarrow \arg\min_j \left\|\boldsymbol{m}_i - (R_k \boldsymbol{z}_j + \boldsymbol{p}_k)\right\|_2^2$$

We assume equal weights w_{ij} is set to 1. With this data association, an optimal rotation $R \in SO(3)$ and translation $p \in \mathbb{R}^3$ are computed using the Kabsch algorithm. This involves computing the centroids of both the source and the target points, mean-centering the points $(\bar{z}, \bar{m})$, and constructing the Q matrix. With Singular Value Decomposition (SVD),

$$Q = U \Sigma V^T$$

For strictly orthogonal and reflection free rotation, a correction matrix $\text{diag}(1, 1, \det(UV^T))$ is used. Finally rotation R and translation p is computed.

$$R = U \text{diag}(1, 1, \det(UV^T)) V^T$$

$$p = \bar{z} - R\bar{m}$$

To avoid being stuck at a local optima, which depends on the initial pose estimate, we assume the primary axis of rotation is the z-axis. The initial yaw angle is discretized from $0°$ to $360°$ in $30°$ increments. For each discretized initialization, The pose transform with the least MSE is chosen as the optimal 3-D pose.

2) Scan Matching: To account for the inherent drift accumulation in dead-reckoning, consecutive LiDAR scans are aligned using a 2D ICP algorithm. Raw LiDAR measurements from the robot are provided as ranges and beam angles. These are converted into 2D Cartesian point clouds. The raw LiDAR measurements from beyond 0.5m were filtered to mitigate sensor noise and discard invalid readings.

ICP algorithm is sensitive to its initial alignment and prone to local minima convergence. To address this, the initial relative transform between consecutive scans is set using the previously computed odometry model. For two consecutive LiDAR timestamps, the closest corresponding odometry timestamps are identified. The relative odometry transform, i.e, ${}_tT_{t+1}^{odo} = (T_t^{odo})^{-1}T_{t+1}^{odo}$, is used as the initial guess for the ICP algorithm.

At each ICP iteration, an association between two consecutive scans are made. A 2-D variation of the Kabsch algorithm is used with the association, to estimate the optimal R and p, similar to the warm-up implementation. This exercise is repeated until convergence, when the MSE of matched points fall below 10^{-5}. This optimized relative pose change ${}_tT_{t+1}$ is used to update the robot's global trajectory. Starting from an initial identity pose T_0, the global pose at time $t+1$ is updated in a iterative manner such that $T_{t+1} = T_t \cdot {}_tT_{t+1}$.

C. Occupancy & Texture Mapping

1) Occupancy Grid Mapping: To build a spatial representation of the environment, a 2-D probabilistic occupancy grid map is constructed. The environment is mapped to a finite grid with a resolution of 0.025 meters per cell, across 60×60 canvas (±30 meters from the initial origin). Each cell in the grid stores a log-odds value representing the posterior probability of containing an obstacle. They are initialized to zero to represent unknown state at the start.

Valid scan points from the LiDAR are transformed to world frame. Based on the world frame coordinates x_{world}, we map them to discrete grid index x_{grid} via $x_{grid} = \lfloor(x_{world} + \text{bounds})/\text{resolution}\rfloor$, where bounds is 30 and resolution is 0.025 as we discussed. The robot's position and the terminal obstacle hit-points are mapped to the grid in this manner.

To execute the probabilistic update, a ray-casting operation is performed for every valid LiDAR beam using a 2-D Bresenham's line algorithm. This identifies the precise array of grid indices that exist in a linear path between the robot's current cell and the terminal hit cell. Every grid cell in this linear path is classified as free space, and its log-odds value is updated by adding $-\log(9)$. The terminal cell corresponding to the obstacle is marked as occupied, and its log-odds value is updated by adding $\log(9)$. This is recursively applied across the entirety of the trajectory and scans to construct the final, dense occupancy grid map of the environment.

2) Textured Occupancy Grid: We fuse the RGBD camera data with the optimized robot trajectory to get a 2-D color texture map of the floor. The texture map is initialized as a 3-channel RGB grid array with the same physical resolution of 0.025 meters per cell as the occupancy map $C \in \mathbb{R}^{N\times N\times 3}$. Since the Kinect data logging occurs at irregular intervals, we achieve synchronization by associating each disparity image & RGB image timestamp with the closest SLAM pose timestamp.

The depth camera is situated at a translation of $p_{cam} = [0.18, 0.005, 0.36]^T$ meters relative to the robot's geometric center, with an orientation represented in $eulerangles = [0, 0.36, 0.021]^T$ as roll, pitch, yaw respectively. Using these extrinsic parameters, alongside a fixed rotation to align the camera optical axes (Z-forward, X-right, Y-down) with the robot's body frame (X-forward, Y-left, Z-up), a constant 4×4 homogeneous transformation matrix T_{cam} is constructed.

We also update, the 2-D optimized robot pose $T_t \in SE(2)$ to a 3-D transformation matrix $T_{body_to_world} \in SE(3)$ taking planar motion ($Z = 0$, roll $= 0$, pitch $= 0$).

The raw disparity value d at pixel location (u, v) is converted to depth using the relations

$$dd = -0.00304d + 3.31 \qquad \text{depth} = 1.03/dd$$

$$K = \begin{bmatrix} fs_u & fs_\theta & c_u \\ 0 & fs_v & c_v \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 585.05 & 0 & 242.94 \\ 0 & 585.05 & 315.84 \\ 0 & 0 & 1 \end{bmatrix}$$

Utilizing the camera's intrinsic matrix K, each pixel (u, v) is projected into the 3-D optical frame to obtain coordinates $(x_c, y_c, depth)$. These local coordinates are then transformed into global world coordinates by sequentially applying the camera extrinsic transformation and the robot pose.

$$P_{world} = T_{body_to_world}T_{cam}P_{cam}$$

Points satisfying an absolute world Z-coordinate of less than 0.05 meters ($|Z_{world}| < 0.05$) and a maximum optical depth of less than 3.0 meters are retained. The corresponding color coordinates

$$rgbi = (526.37i + 19276 - 7877.07dd)/585.051$$

$$rgbj = (526.37j + 16662)/585.051$$

Finally, we convert them to discrete grid indices, and assign the corresponding RGB values to the 2-D texture grid array.

D. Pose graph optimization and loop closure

To refine the trajectory estimated by sequential scan matching and to correct residual cumulative drift, a pose graph optimization is performed using the Georgia Tech Smoothing and Mapping (GTSAM) library. The trajectory is modeled as a nonlinear factor graph where each node represents a 2-D robot pose. The graph is initialized with high-confidence prior factor on the first pose to anchor the trajectory start. The subsequent relative transformations obtained are encoded as odometry factors connecting consecutive pose nodes, utilizing a diagonal Gaussian noise model to represent the inherent measurement uncertainty. Diagonal noise is used under the assumption that any and all estimate noise are independent.

We use two distinct loop closure strategies. First, a fixed-interval loop closure that evaluates every 10 poses by attempting to register the current LiDAR scan with the scan recorded exactly 10 steps prior. Second, a proximity-based search is executed at each step to identify past poses—excluding the

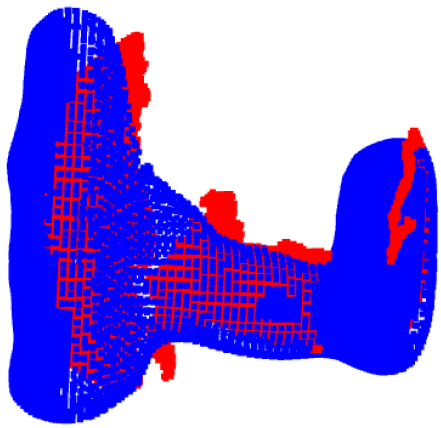

Fig. 1. **Drill - 1**: 3D ICP Visualization of the Drill model

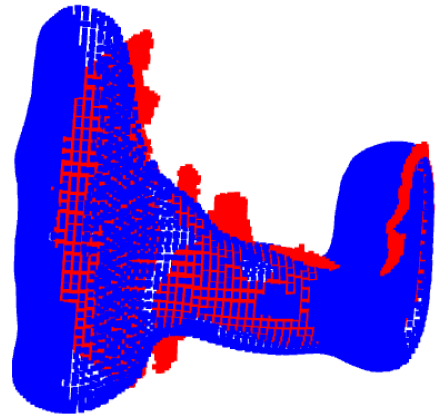

Fig. 2. **Drill - 2**: 3D ICP Visualization of the Drill model

most recent ones, that fall within a Euclidean distance of 1.0 meter from the current pose estimate.

A loop closure edge is only added to the factor graph if the mean squared error (MSE) of the ICP correspondences falls below a strict tolerance threshold of 0.5. Once the graph is fully populated, the GTSAM optimizer is used to minimize the nonlinear least-squares discrepancy. We use the Gauss-Newton method as discussed in class to solve the optimization problem, which is formulated as:

$$f(x) = \frac{1}{2} e^T(x) e(x)$$

The Gauss-Newton update at some iteration k, with varying step size α_k, is given by:

$$x^{k+1} \leftarrow x^k + \alpha_k \delta x$$

Here, the descent direction δx is determined by solving the following relation:

$$\left(\frac{\partial e(x)}{\partial x}\Big|_{x=x_k} \right)^T \left(\frac{\partial e(x)}{\partial x}\Big|_{x=x_k} \right) \delta x = - \left(\frac{\partial e(x)}{\partial x}\Big|_{x=x_k} \right)^T e(x_k)$$

Finally, the optimized poses are extracted. These refined poses are then used to once again map the occupancy grid and texture map.

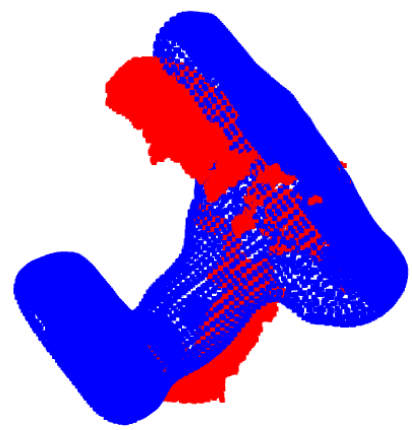

Fig. 3. **Drill - 3**: 3D ICP Visualization of the Drill model

IV. Results

A. 3D ICP Warm-Up

The 3D ICP visualizations show successful registration, with the partial source point clouds accurately mapping onto the surface geometries of the dense target models. The algorithm proves robust across varied physical shapes, tightly aligning to both the complex, multi-angled structure of the drill and the smooth, continuous curves of the liquid bottle. The results are seen from Fig 1 - 8.

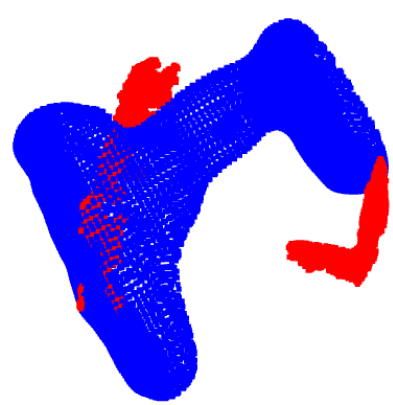

Fig. 4. **Drill - 4**: 3D ICP Visualization of the Drill model

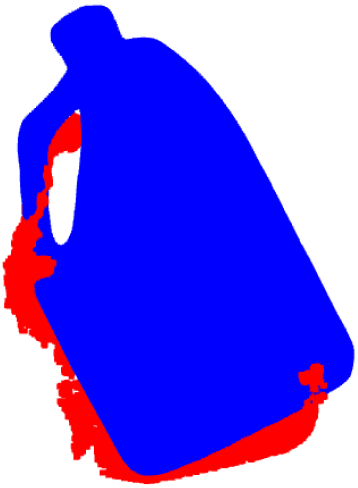

Fig. 5. **Liquid - 1**: 3D ICP Visualization of the Liquid Container model

Fig. 6. **Liquid - 2**: 3D ICP Visualization of the Liquid Container model

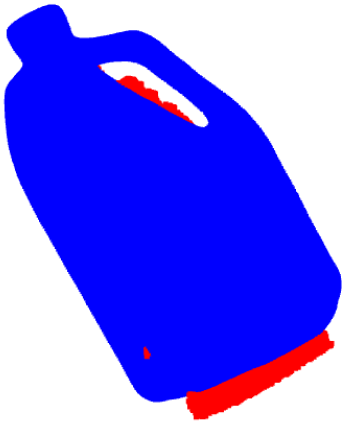

Fig. 7. **Liquid - 3**: 3D ICP Visualization of the Liquid Container model

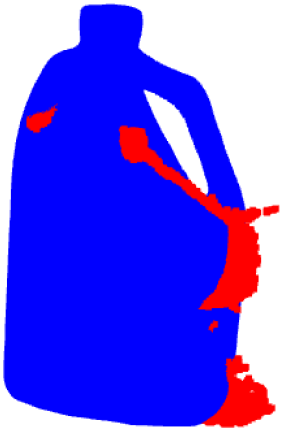

Fig. 8. **Liquid - 4**: 3D ICP Visualization of the Liquid Container model

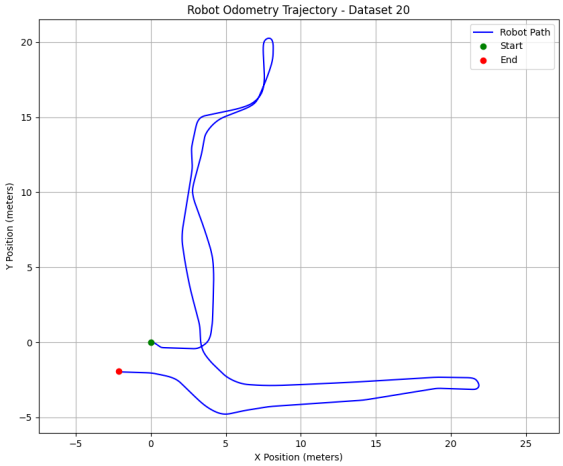

Fig. 9. **Dataset - 20**: IMU & Encoder integration, Robot Trajectory

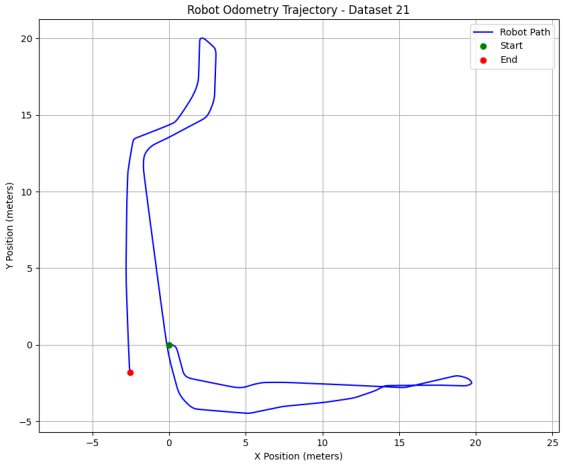

Fig. 10. **Dataset - 21**: IMU & Encoder integration, Robot Trajectory

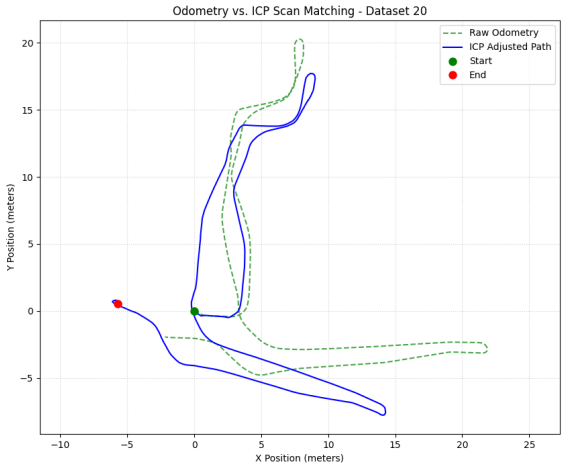

Fig. 11. **Dataset - 20**: Raw vs ICP, Robot Trajectory

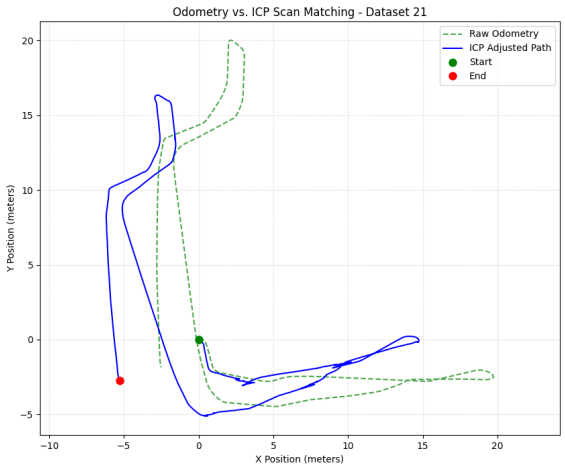

Fig. 12. **Dataset - 21**: Raw vs ICP, Robot Trajectory

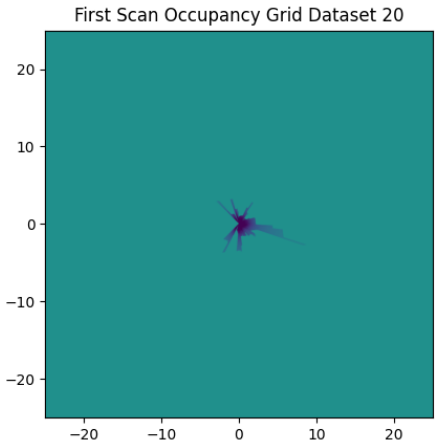

Fig. 13. **Dataset - 20**: LiDAR first scan occupancy

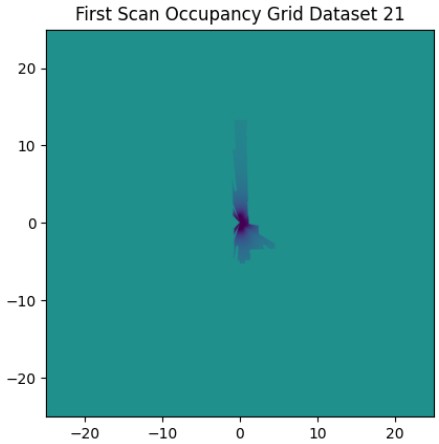

Fig. 14. **Dataset - 21**: LiDAR first scan occupancy

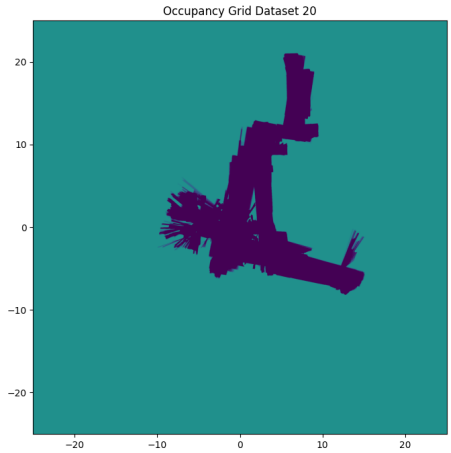

Fig. 15. **Dataset - 20**: Full Occupancy grid from LiDAR scans

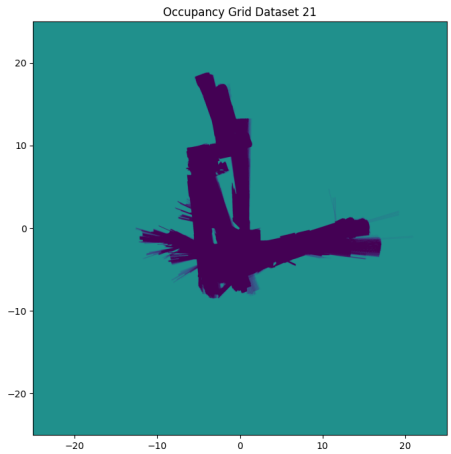

Fig. 16. **Dataset - 21**: Full Occupancy grid from LiDAR scans

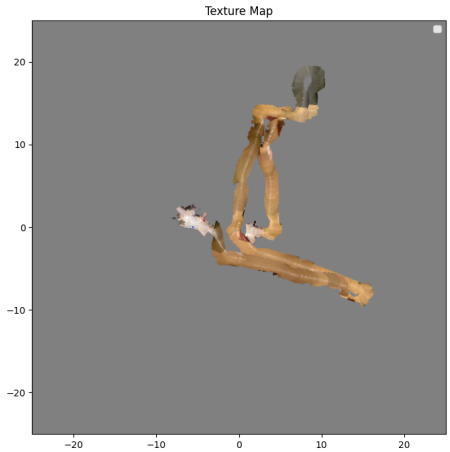

Fig. 17. **Dataset - 20**: Textured Occupancy grid from RGBD

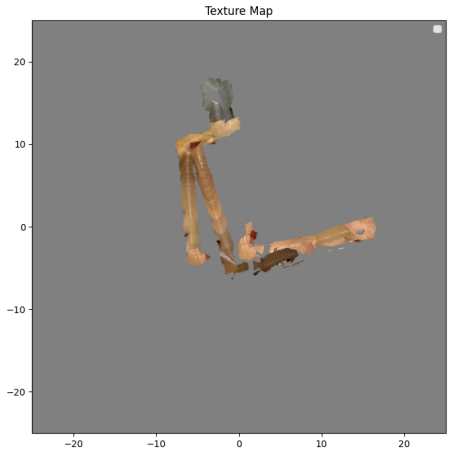

Fig. 18. **Dataset - 21**: Textured Occupancy grid from RGBD

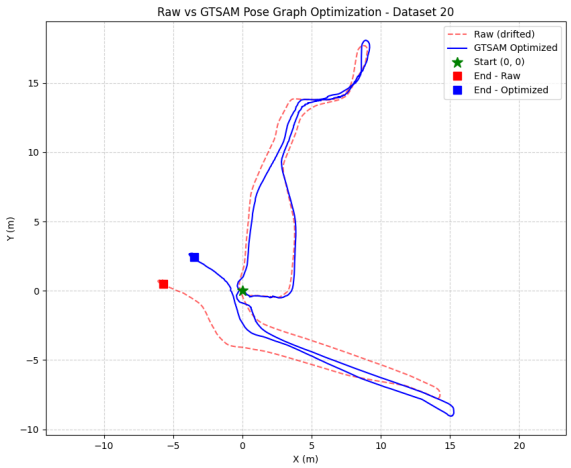

Fig. 19. **Dataset - 20**: GTSAM optimized vs Raw Robot Trajectory

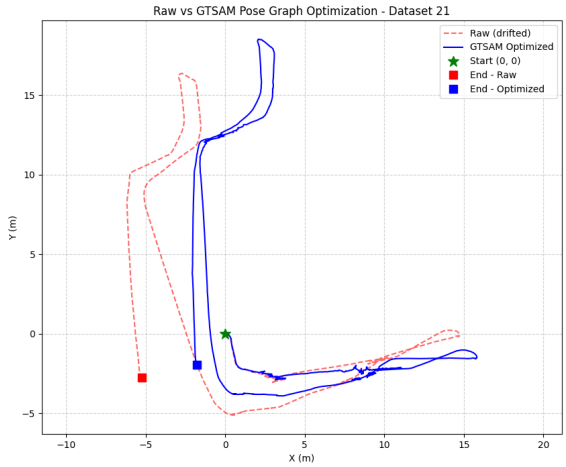

Fig. 20. **Dataset - 21**: GTSAM optimized vs Raw Robot Trajectory

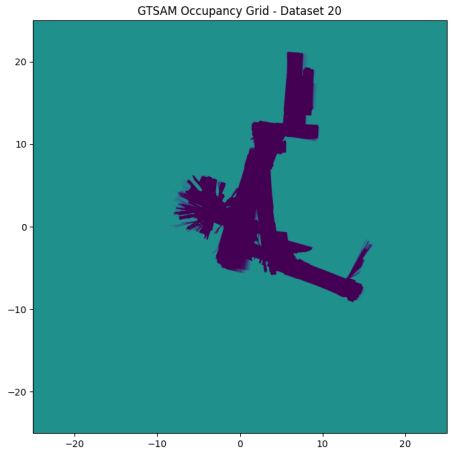

Fig. 21. **Dataset - 20**: GTSAM optimized Occupancy grid from LiDAR scans

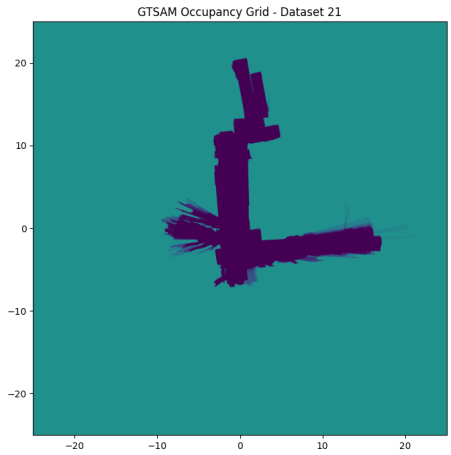

Fig. 22. **Dataset - 21**: GTSAM optimized Occupancy grid from LiDAR scans

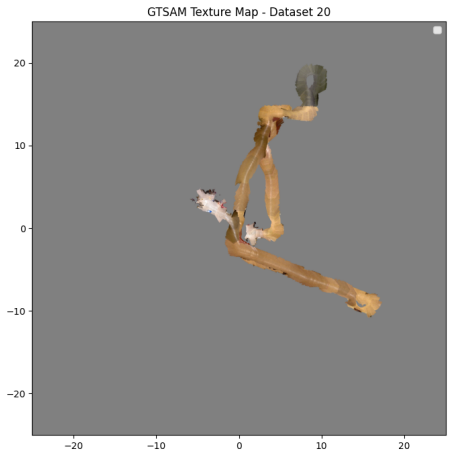

Fig. 23. **Dataset - 20**: GTSAM optimized Texture map from RGBD

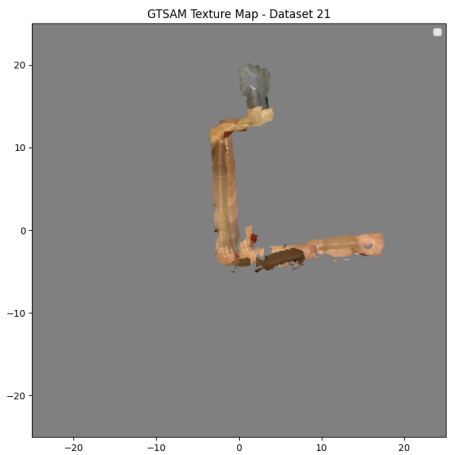

Fig. 24. **Dataset - 21**: GTSAM optimized Texture map from RGBD

B. Robot Trajectory

We see the Raw odometry trajectory generated from IMU and Wheel Encoders in Fig 9 and Fig 10. We get a smooth trajectory, however, there could be sensor drift that causes the trajectory to be accurate. We proceed to use advanced techniques.

Fig 11 and Fig 12, show the raw odometry trajectory and ICP supported trajectory. The most noticeable difference between the two is the change in heading $theta$. As seen, the change in heading gets more drastic.

Fig 19 and 20 illustrate the robot's trajectory before and after GTSAM pose graph optimization. Initially, the raw odometry suggests the robot took slightly different paths during its forward and return journeys. The difference in the paths strongly indicates accumulated rotational and translational sensor drift rather than a true physical divergence. By utilizing proximity and fixed-interval loop closure constraints, our factor graph optimization was able to minimize this error. The optimized trajectory successfully snaps both the forward and backward paths close, confirming the robot accurately traversed the same hallway in both directions. This corrected, globally consistent trajectory is critical for generating crisp, texture and occupancy maps in the final pipeline downstream. While the optimized trajectory is slightly jagged, it still offers superior pose estimates.

C. Occupancy Grid

The improvements in trajectory directly translate to a more accurate Occupancy Grid. The most notable correction is observed in Dataset 21. In the non-optimized version (Fig. 16), a double hallway is visible which is a common effect of pose error where the map fails to register that the robot is revisiting a previously mapped area, and also due to sensor data drift.

Following the GTSAM optimization (Fig. 21), this artifact is eliminated, consolidating the map into a single, cohesive corridor. Beyond simple trajectory alignment, the optimized grid displays significantly higher structural fidelity. The boundaries are sharper, and the jagged noise typically caused by inconsistent pose-stamping of LIDAR scans is greatly reduced.

D. Texture Map

The Texture Map (Fig. 17, 18, 20, 21) projects RGB data onto the spatial framework, providing semantic context that the occupancy grid lacks. The primary value of the texture map in this report is to visualize the environmental features which could help in identifying specific landmarks. While the texture maps do not fundamentally change in geometry, they benefit directly from the underlying pose optimization as seen with the Occupancy map.

E. Future Improvements

While GTSAM effectively corrected major structural drift, some minor challenges persisted. Strictly enforcing the ICP MSE tolerance threshold of 0.5 resulted in the rejection of some potentially valid loop closures, highlighting a delicate trade-off between maximizing trajectory smoothness and preventing false-positive associations. For a smoother trajectory, especially for Dataset - 21, dropping points farther than 0.5m could be an potential place for additional fine-tuning.

V. Conclusion

This project successfully implemented LiDAR-based SLAM and trajectory optimization using ICP and GTSAM-based loop closures. While the initial ICP scan matching provided a foundational odometry estimate, it was inherently susceptible to accumulated rotational and translational drift. By constructing a pose graph and enforcing both fixed-interval and proximity-based loop closures, GTSAM effectively minimized this error and restored global consistency to the robot's trajectory. We also leveraged an RGBD sensor to add texture to the occupancy map. Ultimately, this project highlights the need of graph optimization in transforming raw, localized sensor data into reliable models.

VI. Reference

[1] https://github.com/borglab/gtsam
[2] https://natanaso.github.io/ece276a/ref/ECE276A/4/MotionAndObservationModels.pdf
[3] https://natanaso.github.io/ece276a/ref/ECE276A/5/FactorGraphSLAM.pdf
[4] https://natanaso.github.io/ece276a/ref/ECE276A/6/LocalizationOdometry.pdf